

Deep Learning

Vazgen Mikayelyan

June 5, 2021



Receptive Field

Let we have a convolutional neural network with consequent layers $F_0, F_1, \dots, F_D$ (F_0, F_D are inputs and outputs of the network respectively).

Receptive Field

Let we have a convolutional neural network with consequent layers $F_0, F_1, \dots, F_D$ (F_0, F_D are inputs and outputs of the network respectively).

Receptive Field

The Receptive Field of a layer F_k with respect to the layer F_m ($m < k$) is the maximal region in the layer F_m , each element of which is contributed in forming one of pixels in F_k , considering only convolutional layers as contribution.

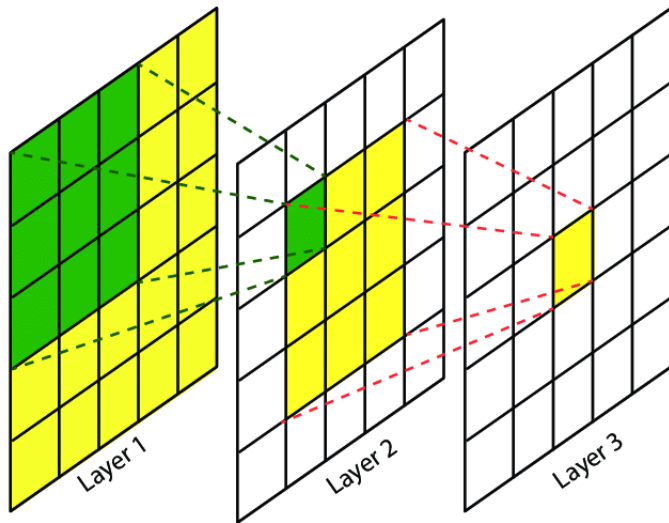
Receptive Field

Let we have a convolutional neural network with consequent layers $F_0, F_1, \dots, F_D$ (F_0, F_D are inputs and outputs of the network respectively).

Receptive Field

The Receptive Field of a layer F_k with respect to the layer F_m ($m < k$) is the maximal region in the layer F_m , each element of which is contributed in forming one of pixels in F_k , considering only convolutional layers as contribution. By receptive field of a network we mean the receptive field of its output with respect to its input.

Receptive Field



Remark 1

Although the receptive field can be referred as the "area under vision of the net", not all pixels in receptive field are "equally contributed" in formation of an output pixel. So there is a concept of effective receptive field.

Remark 1

Although the receptive field can be referred as the "area under vision of the net", not all pixels in receptive field are "equally contributed" in formation of an output pixel. So there is a concept of effective receptive field.

So the natural need arises to enlarge the size of the receptive field of the network, or keep the size of receptive field the same but decrease the number of computations. For the latter purpose the approach of convolution factorization is appropriate.

Convolution Factorization

The concept of convolution factorization was introduced in the Inception-V3 architecture.

Convolution Factorization

The concept of convolution factorization was introduced in the Inception-V3 architecture. The idea is the following: in some places

- replace 5×5 convolution layers with two 3×3 convolution layers,

Convolution Factorization

The concept of convolution factorization was introduced in the Inception-V3 architecture. The idea is the following: in some places

- replace 5×5 convolution layers with two 3×3 convolution layers,
- replace $n \times n$ convolution layers with asymmetric consequent convolution layers of sizes $n \times 1$ and $1 \times n$.

Convolution Factorization

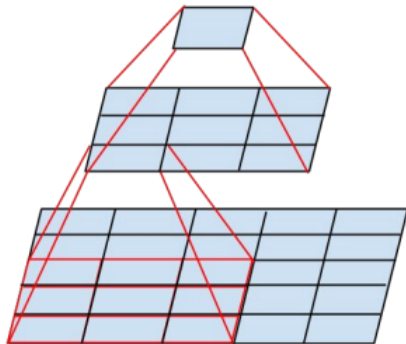


Figure 1. Mini-network replacing the 5×5 convolutions.

Convolution Factorization

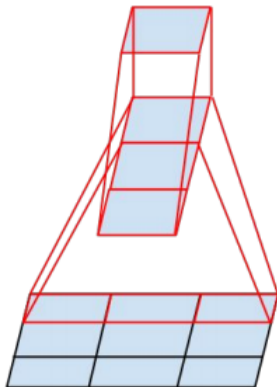


Figure 3. Mini-network replacing the 3×3 convolutions. The lower layer of this network consists of a 3×1 convolution with 3 output units.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.
- Inception v2 (2015) has approximately 11.2M parameters.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.
- Inception v2 (2015) has approximately 11.2M parameters.
- Accuracies on ImageNet: Top1=74.8%, Top5=92.2%.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.
- Inception v2 (2015) has approximately 11.2M parameters.
- Accuracies on ImageNet: Top1=74.8%, Top5=92.2%.
- Inception v3 (2015) has approximately 23.8M parameters.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.
- Inception v2 (2015) has approximately 11.2M parameters.
- Accuracies on ImageNet: Top1=74.8%, Top5=92.2%.
- Inception v3 (2015) has approximately 23.8M parameters.
- Accuracies on ImageNet: Top1=78.8%, Top5=94.4%.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.
- Inception v2 (2015) has approximately 11.2M parameters.
- Accuracies on ImageNet: Top1=74.8%, Top5=92.2%.
- Inception v3 (2015) has approximately 23.8M parameters.
- Accuracies on ImageNet: Top1=78.8%, Top5=94.4%.
- Inception v4/Inception-ResNet (2016) has approximately 55.8M parameters.

- GoogLeNet/Inception v1 (2014) has approximately 5M parameters.
- Accuracies on ImageNet: Top1=69.8%, Top5=89.9%.
- Inception v2 (2015) has approximately 11.2M parameters.
- Accuracies on ImageNet: Top1=74.8%, Top5=92.2%.
- Inception v3 (2015) has approximately 23.8M parameters.
- Accuracies on ImageNet: Top1=78.8%, Top5=94.4%.
- Inception v4/Inception-ResNet (2016) has approximately 55.8M parameters.
- Accuracies on ImageNet: Top1=80.1%, Top5=95.1%.

Depthwise Separable Convolution

Depthwise Separable Convolution

- Depthwise separable convolution is a form of factorized convolution which factorize a standard convolution into a depthwise convolution and a 1×1 convolution called a pointwise convolution.

Depthwise Separable Convolution

- Depthwise separable convolution is a form of factorized convolution which factorize a standard convolution into a depthwise convolution and a 1×1 convolution called a pointwise convolution.
- The depthwise convolution applies a single filter to each input channel.

Depthwise Separable Convolution

- Depthwise separable convolution is a form of factorized convolution which factorize a standard convolution into a depthwise convolution and a 1×1 convolution called a pointwise convolution.
- The depthwise convolution applies a single filter to each input channel.
- The pointwise convolution then applies a 1×1 convolution to combine the outputs the depthwise convolution.

Depthwise Separable Convolution

- Depthwise separable convolution is a form of factorized convolution which factorize a standard convolution into a depthwise convolution and a 1×1 convolution called a pointwise convolution.
- The depthwise convolution applies a single filter to each input channel.
- The pointwise convolution then applies a 1×1 convolution to combine the outputs the depthwise convolution.
- So the depthwise separable convolution splits this into two layers, a separate layer for filtering and a separate layer for combining.

Depthwise Separable Convolution

- Depthwise separable convolution is a form of factorized convolution which factorize a standard convolution into a depthwise convolution and a 1×1 convolution called a pointwise convolution.
- The depthwise convolution applies a single filter to each input channel.
- The pointwise convolution then applies a 1×1 convolution to combine the outputs the depthwise convolution.
- So the depthwise separable convolution splits this into two layers, a separate layer for filtering and a separate layer for combining.
- This factorization has the effect of drastically reducing computation and model size.

Depthwise Separable Convolution

Let we have a feature map with shape $D_F \times D_F \times M$ and we want to produce a $D_F \times D_F \times N$ feature map.

Depthwise Separable Convolution

Let we have a feature map with shape $D_F \times D_F \times M$ and we want to produce a $D_F \times D_F \times N$ feature map. In the case of standard convolutions with the shape $D_K \times D_K \times M \times N$ the number of multiplications will be:

Depthwise Separable Convolution

Let we have a feature map with shape $D_F \times D_F \times M$ and we want to produce a $D_F \times D_F \times N$ feature map. In the case of standard convolutions with the shape $D_K \times D_K \times M \times N$ the number of multiplications will be:

$$D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F.$$

Depthwise Separable Convolution

Let we have a feature map with shape $D_F \times D_F \times M$ and we want to produce a $D_F \times D_F \times N$ feature map. In the case of standard convolutions with the shape $D_K \times D_K \times M \times N$ the number of multiplications will be:

$$D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F.$$

In the case of depthwise separable convolution, the number of multiplications will be:

Depthwise Separable Convolution

Let we have a feature map with shape $D_F \times D_F \times M$ and we want to produce a $D_F \times D_F \times N$ feature map. In the case of standard convolutions with the shape $D_K \times D_K \times M \times N$ the number of multiplications will be:

$$D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F.$$

In the case of depthwise separable convolution, the number of multiplications will be:

$$D_K \cdot D_K \cdot M \cdot D_F \cdot D_F +$$

Depthwise Separable Convolution

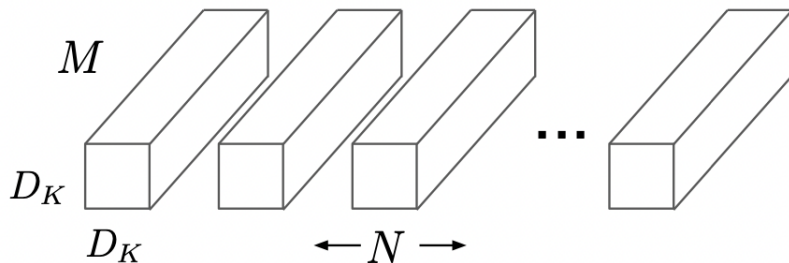
Let we have a feature map with shape $D_F \times D_F \times M$ and we want to produce a $D_F \times D_F \times N$ feature map. In the case of standard convolutions with the shape $D_K \times D_K \times M \times N$ the number of multiplications will be:

$$D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F.$$

In the case of depthwise separable convolution, the number of multiplications will be:

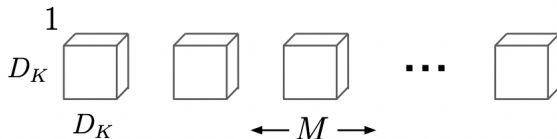
$$D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F.$$

Depthwise Separable Convolution

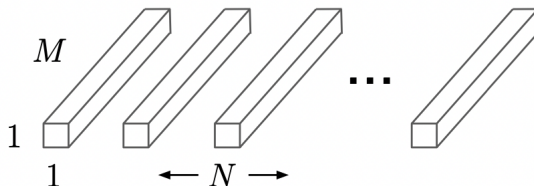


(a) Standard Convolution Filters

Depthwise Separable Convolution



(b) Depthwise Convolutional Filters



(c) 1×1 Convolutional Filters called Pointwise Convolution in the context of Depthwise Separable Convolution

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 1024$
	Conv dw / s2	$3 \times 3 \times 1024$ dw
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
	Avg Pool / s1	Pool 7×7
	FC / s1	1024×1000
	Softmax / s1	Classifier

MobileNet (2017)

- MobileNet (2017) has approximately 4.2M parameters.

MobileNet (2017)

- MobileNet (2017) has approximately 4.2M parameters.
- Accuracies on ImageNet: Top1=70.6%, Top5=89.5%.

MobileNet (2017)

- MobileNet (2017) has approximately 4.2M parameters.
- Accuracies on ImageNet: Top1=70.6%, Top5=89.5%.
- The best result on ImageNet: Top1=88.5%, Top5=98.7% with 480M parameters.

1 Transfer Learning

Problems of deep learning

- It is difficult to find datasets of huge sizes, and it is way too costly to prepare such datasets.

Problems of deep learning

- It is difficult to find datasets of huge sizes, and it is way too costly to prepare such datasets.
- The high cost of GPUs needed to run advanced deep learning algorithms.

Problems of deep learning

- It is difficult to find datasets of huge sizes, and it is way too costly to prepare such datasets.
- The high cost of GPUs needed to run advanced deep learning algorithms.
- Training takes a long time.

Idea of transfer learning

Instead of training a deep network from scratch for your task:

Idea of transfer learning

Instead of training a deep network from scratch for your task:

- Take a network trained on a different domain for a different **source task**.

Idea of transfer learning

Instead of training a deep network from scratch for your task:

- Take a network trained on a different domain for a different **source task**.
- Adapt it for your domain and your **target task**.

Idea of transfer learning

Instead of training a deep network from scratch for your task:

- Take a network trained on a different domain for a different **source task**.
- Adapt it for your domain and your **target task**.

Variations:

- Same domain, different task.
- Different domain, same task.

Idea of transfer learning

When transfer learning makes sense?

Idea of transfer learning

When transfer learning makes sense?

- Task A and B have the same input x .

Idea of transfer learning

When transfer learning makes sense?

- Task A and B have the same input x .
- You have a lot of more data for Task A than Task B.

Idea of transfer learning

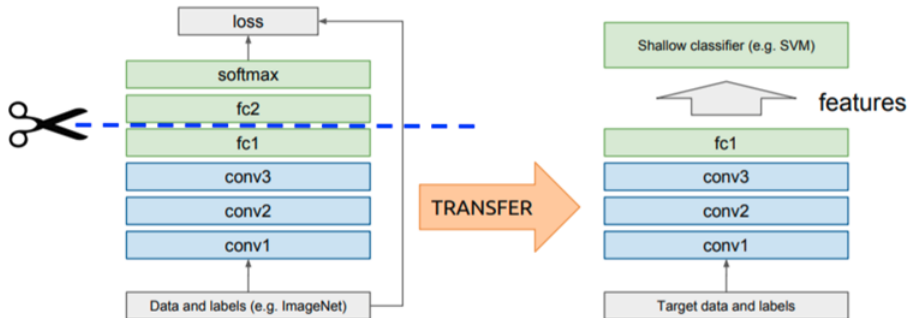
When transfer learning makes sense?

- Task A and B have the same input x .
- You have a lot of more data for Task A than Task B.
- Low level features from A could be helpful for learning B.

Fixed feature extractor

For example take a NN pretrained on some big dataset, remove the last fully-connected layer, then treat the rest of the NN as a fixed feature extractor for the new dataset. Then train a classifier (e.g. Linear SVM or Softmax classifier) for the new dataset.

Fixed feature extractor



This strategy is to not only replace and retrain the classifier on top of the deep net on the new dataset, but to also fine-tune the weights of the pretrained network by continuing the backpropagation.

This strategy is to not only replace and retrain the classifier on top of the deep net on the new dataset, but to also fine-tune the weights of the pretrained network by continuing the backpropagation.

Variations:

This strategy is to not only replace and retrain the classifier on top of the deep net on the new dataset, but to also fine-tune the weights of the pretrained network by continuing the backpropagation.

Variations:

- Fine-tune all the layers of the network.

This strategy is to not only replace and retrain the classifier on top of the deep net on the new dataset, but to also fine-tune the weights of the pretrained network by continuing the backpropagation.

Variations:

- Fine-tune all the layers of the network.
- Fine-tune only some higher-level portion of the network.

This strategy is to not only replace and retrain the classifier on top of the deep net on the new dataset, but to also fine-tune the weights of the pretrained network by continuing the backpropagation.

Variations:

- Fine-tune all the layers of the network.
- Fine-tune only some higher-level portion of the network.
- Fine-tune all the layers of the network with different learning rates.

This strategy is to not only replace and retrain the classifier on top of the deep net on the new dataset, but to also fine-tune the weights of the pretrained network by continuing the backpropagation.

Variations:

- Fine-tune all the layers of the network.
- Fine-tune only some higher-level portion of the network.
- Fine-tune all the layers of the network with different learning rates.

The last two points are motivated by the observation that the earlier features of the network contain more generic features.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset?

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.
- New dataset is large and similar to the original dataset.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.
- New dataset is large and similar to the original dataset.
Fine-tune the whole network.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.
- New dataset is large and similar to the original dataset.
Fine-tune the whole network.
- New dataset is small but very different from the original dataset.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.
- New dataset is large and similar to the original dataset.
Fine-tune the whole network.
- New dataset is small but very different from the original dataset.
Use SVM without transfer learning.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.
- New dataset is large and similar to the original dataset.
Fine-tune the whole network.
- New dataset is small but very different from the original dataset.
Use SVM without transfer learning.
- New dataset is small and similar to original dataset.

When and how to fine-tune?

How do you decide what type of transfer learning you should perform on a new dataset? This is a function of several factors, but the two most important ones are the size of the new dataset, and its similarity to the original dataset.

Scenarios:

- New dataset is large and very different from the original dataset.
Train from scratch the whole network.
- New dataset is large and similar to the original dataset.
Fine-tune the whole network.
- New dataset is small but very different from the original dataset.
Use SVM without transfer learning.
- New dataset is small and similar to original dataset.
Fine-tune only some higher-level portion of the network.

Transfer learning in image specific tasks



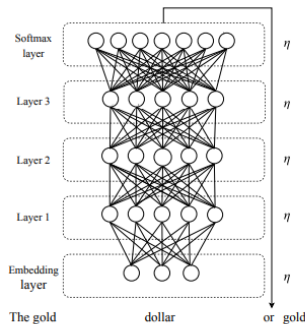
Transfer learning in NLP

The first successful transfer learning in NLP was done in 2018 which surpassed all text classification state-of-the-art. There was created framework ULMFiT, for transfer learning tasks in NLP.

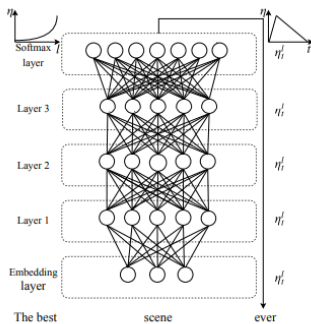
Transfer learning in NLP

The first successful transfer learning in NLP was done in 2018 which surpassed all text classification state-of-the-art. There was created framework ULMFiT, for transfer learning tasks in NLP.

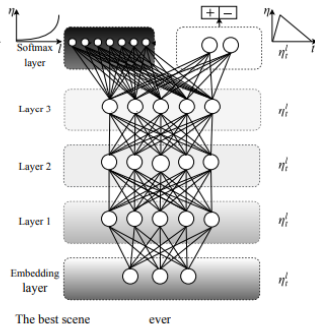




(a) LM pre-training



(b) LM fine-tuning



(c) Classifier fine-tuning

Main ideas in ULMFiT

- Discriminative fine tuning

- Discriminative fine tuning

Different learning rates are used for different layers during the fine-tuning phase of LM (on the target task). This is done because the layers capture different types of information.

- Discriminative fine tuning

Different learning rates are used for different layers during the fine-tuning phase of LM (on the target task). This is done because the layers capture different types of information.

- Slanted triangular learning rates (STLR)

- Discriminative fine tuning

Different learning rates are used for different layers during the fine-tuning phase of LM (on the target task). This is done because the layers capture different types of information.

- Slanted triangular learning rates (STLR)

Learning rates are first increased linearly, and then decreased gradually after a cut, i.e., there is a “short increase” and a “long decay.”

- Discriminative fine tuning

Different learning rates are used for different layers during the fine-tuning phase of LM (on the target task). This is done because the layers capture different types of information.

- Slanted triangular learning rates (STLR)

Learning rates are first increased linearly, and then decreased gradually after a cut, i.e., there is a “short increase” and a “long decay.”

- Gradual unfreezing

- Discriminative fine tuning

Different learning rates are used for different layers during the fine-tuning phase of LM (on the target task). This is done because the layers capture different types of information.

- Slanted triangular learning rates (STLR)

Learning rates are first increased linearly, and then decreased gradually after a cut, i.e., there is a “short increase” and a “long decay.”

- Gradual unfreezing

During the classification training, the LM model is gradually unfreezed starting from the last layer.

